

GlicoCare - Specifiche per Class Diagram UML

Generato il: 30/06/2026 12:28

Contiene la definizione completa di Classi, Attributi e Metodi per la progettazione del diagramma (inclusi costruttori e metodi privati).

■ Modulo: main.py

Percorso: `main.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: user.py

Percorso: `src/user.py`

■■ Classe: CredenzialiNonValide

Descrizione: Eccezione sollevata quando le credenziali di accesso non sono valide.

Estende: Exception, BaseException

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
args	Any	
kwargs	Any	

■ Costruttore:

+ **CredenzialiNonValide(self, /, *args, **kwargs)**

Descrizione: Initialize self. See help(type(self)) for accurate signature.

■ Metodi (UML: + pubblico, - privato):

Nessun metodo (oltre al costruttore) definito.

■■ Classe: User

Descrizione: Rappresenta un utente del sistema e gestisce l'autenticazione. Args: username (str): Nome utente da autenticare. password (str): Password in chiaro da verificare. db_path (str, optional): Percorso del file del database SQLite. Default: "glicocare.db". Raises: CredenzialiNonValide: Se le credenziali fornite non sono corrette.

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
username	<class 'str'>	Nome utente da autenticare.
password	<class 'str'>	Password in chiaro da verificare.
db_path	<class 'str'>	Percorso del file del database SQLite.

■ Costruttore:

+ User(self, username: str, password: str, db_path: str = 'glicocare.db')

Descrizione: Initialize self. See help(type(self)) for accurate signature.

■ Metodi (UML: + pubblico, - privato):

■ - __str__

Descrizione: Restituisce una rappresentazione leggibile dell'utente.

Firma: __str__(self) -> str

Ritorna: str: Stringa riassuntiva dell'utente.

■ - _get_connection

Descrizione: Crea e restituisce una connessione al database.

Firma: _get_connection(self) -> sqlite3.Connection

Ritorna: sqlite3.Connection: Oggetto connessione al database.

■ - _hash_password

Descrizione: Calcola l'hash SHA-256 di una password.

Firma: _hash_password(password: str) -> str

Ritorna: password (str): Password in chiaro. str: Hash della password in formato esadecimale.

■ - _verifica_credenziali

Descrizione: Verifica le credenziali dell'utente nel database. Se la verifica ha successo, gli attributi `tipo` e `id\_ref` vengono popolati con i valori letti dal database.

Firma: _verifica_credenziali(self, password: str) -> bool

Ritorna: password (str): Password in chiaro da verificare. bool: True se le credenziali sono valide, False altrimenti.

■ + get_user_info

Descrizione: Restituisce un dizionario con le informazioni dell'utente autenticato.

Firma: get_user_info(self) -> dict

Ritorna: dict: Dizionario con chiavi 'username', 'tipo' e 'id_ref'.

■ + is_medico

Descrizione: Verifica se l'utente è un medico.

Firma: is_medico(self) -> bool

Ritorna: bool: True se l'utente è di tipo 'M', False altrimenti.

■ + is_paziente

Descrizione: Verifica se l'utente è un paziente.

Firma: is_paziente(self) -> bool

Ritorna: bool: True se l'utente è di tipo 'P', False altrimenti.

■ Modulo: paziente.py

Percorso: `src/paziente.py`

■ ■ Classe: Paziente

Descrizione: Rappresenta un paziente e fornisce metodi per interagire con i suoi dati.

Args: id_ref (int): L'ID del paziente (corrisponde a `id\_paz` nella tabella PAZIENTE).

db_path (str, optional): Percorso del file del database SQLite. Default: "database/glicocare.db".

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
id_ref	Any	L'ID del paziente (corrisponde a `id_paz` nella tabella PAZIENTE).
db_path	Any	Percorso del file del database SQLite.

■ Costruttore:

+ Paziente(self, id_ref, db_path='database/glicocare.db')

Descrizione: Initialize self. See help(type(self)) for accurate signature.

■ Metodi (UML: + pubblico, - privato):

■ - __str__

Descrizione: Restituisce una rappresentazione leggibile del paziente.

Firma: __str__(self) -> str

Ritorna: str: Stringa riassuntiva dei dati del paziente.

■ - _aggiorna_gravita_db

Descrizione: Ricalcola la gravità e la aggiorna nel database e nell'oggetto.

Firma: _aggiorna_gravita_db(self)

■ - _calcola_gravita

Descrizione: Calcola la gravità del paziente: - Ogni valore glicemico fuori soglia = 1 punto

- Ogni giorno con rilevazioni ma senza assunzioni complete = 1 punto

Firma: _calcola_gravita(self)

Ritorna: tuple: (punteggio_totale, punti_glicemia, punti_terapia)

■ - _carica_dati

Descrizione: Carica i dati anagrafici, le rilevazioni e le terapie del paziente dal DB. Questo metodo è privato e viene chiamato automaticamente dal costruttore.

Firma: _carica_dati(self)

■ - _ordina_rilevazioni

Descrizione: Ordina le rilevazioni per giorno decrescente e ora crescente. Risultato: giorno più recente in cima, ore nello stesso giorno dalla più mattutina alla più serale.

Firma: _ordina_rilevazioni(self)

■ - _refresh_rilevazioni

Descrizione: Ricarica la lista delle rilevazioni dal database dopo una modifica.

Firma: _refresh_rilevazioni(self)

■ + aggiornaAssunzione

Descrizione: Aggiorna un'assunzione esistente.

Firma: aggiornaAssunzione(self, vecchio_giorno: str, vecchia_ora: str, vecchio_farmaco: str, nuovo_giorno: str, nuova_ora: str, farmaco: str, quantita: str) -> None

Nome	Tipo	Descrizione
vecchio_giorno	str	Data originale ('YYYY-MM-DD').
vecchia_ora	str	Ora originale ('HH:MM').
vecchio_farmaco	str	Nome originale del farmaco.
nuovo_giorno	str	Nuova data ('YYYY-MM-DD').
nuova_ora	str	Nuova ora ('HH:MM').
farmaco	str	Nuovo nome del farmaco.
quantita	str	Nuova quantità assunta.

■ + aggiornaRilevazioneGiornaliera

Descrizione: Aggiorna una rilevazione della glicemia esistente. Dopo l'aggiornamento, la lista delle rilevazioni viene ricaricata e la gravità del paziente viene ricalcolata.

Firma: aggiornaRilevazioneGiornaliera(self, vecchio_giorno: str, vecchia_ora: str, nuovo_giorno: str, nuova_ora: str, glicemia: float, primaDopoPasto: str) -> None

Nome	Tipo	Descrizione
vecchio_giorno	str	Data originale nel formato 'YYYY-MM-DD'.
vecchia_ora	str	Ora originale nel formato 'HH:MM'.
nuovo_giorno	str	Nuova data nel formato 'YYYY-MM-DD'.
nuova_ora	str	Nuova ora nel formato 'HH:MM'.
glicemia	float	Nuovo valore della glicemia.
primaDopoPasto	str	'P' o 'D'.

■ + aggiornaSegnalazione

Descrizione: Aggiorna una segnalazione esistente.

Firma: aggiornaSegnalazione(self, vecchio_inizio: str, vecchia_fine: str, nuovo_inizio: str, nuova_fine: str, descrizione: str, terapia: str) -> None

Nome	Tipo	Descrizione
vecchio_inizio	str	Data inizio originale ('YYYY-MM-DD').
vecchia_fine	str	Data fine originale ('YYYY-MM-DD').
nuovo_inizio	str	Nuova data inizio ('YYYY-MM-DD').
nuova_fine	str	Nuova data fine ('YYYY-MM-DD').
descrizione	str	Nuova descrizione.
terapia	str	Nuovo farmaco associato (può essere None).

■ + aggiungiAssunzione

Descrizione: Registra un'assunzione di un farmaco da parte del paziente. Verifica che il farmaco sia presente nella terapia del paziente prima di procedere con l'inserimento.

Firma: aggiungiAssunzione(self, giorno: str, ora: str, farmaco: str, quantita: str) -> None

Nome	Tipo	Descrizione
giorno	str	Data dell'assunzione nel formato 'YYYY-MM-DD'.
ora	str	Ora dell'assunzione nel formato 'HH:MM'.

farmaco	str	Nome del farmaco assunto.
quantita	str	Quantità assunta (es. '1 compressa', '5 ml').

■ + aggiungiRilevazioneGiornaliera

Descrizione: Aggiunge una nuova rilevazione della glicemia. Dopo l'inserimento, la lista delle rilevazioni viene ricaricata e la gravità del paziente viene ricalcolata.

Firma: aggiungiRilevazioneGiornaliera(self, giorno: str, ora: str, glicemia: float, primaDopoPasto: str) -> None

Eccezioni sollevate: giorno (str): Data della rilevazione nel formato 'YYYY-MM-DD'. ora (str): Ora della rilevazione nel formato 'HH:MM'. glicemia (float): Valore della glicemia in mg/dL. primaDopoPasto (str): 'P' per prima del pasto, 'D' per dopo il pasto. sqlite3.IntegrityError: Se esiste già una rilevazione con la stessa data e ora.

■ + aggiungiSegnalazione

Descrizione: Aggiunge una nuova segnalazione del paziente.

Firma: aggiungiSegnalazione(self, giorno_inizio: str, giorno_fine: str, descrizione: str, terapia: str = None) -> None

Nome	Tipo	Descrizione
giorno_inizio	str	Data inizio nel formato 'YYYY-MM-DD'.
giorno_fine	str	Data fine nel formato 'YYYY-MM-DD'.
descrizione	str	Descrizione della segnalazione.
terapia	str, optional	Nome del farmaco associato, se presente.

■ + eliminaAssunzione

Descrizione: Elimina un'assunzione esistente.

Firma: eliminaAssunzione(self, giorno: str, ora: str, farmaco: str) -> None

Nome	Tipo	Descrizione
giorno	str	Data dell'assunzione ('YYYY-MM-DD').
ora	str	Ora dell'assunzione ('HH:MM').
farmaco	str	Nome del farmaco assunto.

■ + eliminaRilevazioneGiornaliera

Descrizione: Elimina una rilevazione della glicemia. Dopo l'eliminazione, la lista delle rilevazioni viene ricaricata e la gravità del paziente viene ricalcolata.

Firma: eliminaRilevazioneGiornaliera(self, giorno: str, ora: str) -> None

Nome	Tipo	Descrizione
giorno	str	Data della rilevazione da eliminare ('YYYY-MM-DD').
ora	str	Ora della rilevazione da eliminare ('HH:MM').

■ + eliminaSegnalazione

Descrizione: Elimina una segnalazione esistente.

Firma: eliminaSegnalazione(self, giorno_inizio: str, giorno_fine: str) -> None

Nome	Tipo	Descrizione
giorno_inizio	str	Data inizio della segnalazione ('YYYY-MM-DD').
giorno_fine	str	Data fine della segnalazione ('YYYY-MM-DD').

■ + getAssunzioni

Descrizione: Recupera tutte le assunzioni di farmaci del paziente.

Firma: getAssunzioni(self) -> list

Ritorna: list: Lista di tuple nel formato (giorno (str), ora (str), farmaco (str), quantita (str)).

■ + getCel

Descrizione: Restituisce il numero di cellulare del paziente.

Firma: getCel(self) -> str

■ + getCf

Descrizione: Restituisce il codice fiscale del paziente.

Firma: getCf(self) -> str

■ + getCognome

Descrizione: Restituisce il cognome del paziente.

Firma: getCognome(self) -> str

■ + getDataNascita

Descrizione: Restituisce la data di nascita del paziente.

Firma: getDataNascita(self) -> str

■ + getEmail

Descrizione: Restituisce l'email del paziente.

Firma: getEmail(self) -> str

■ + getGravita

Descrizione: Restituisce il livello di gravità calcolato.

Firma: getGravita(self) -> int

■ + getIdRef

Descrizione: Restituisce l'ID di riferimento del paziente.

Firma: getIdRef(self) -> int

■ + getIndirizzo

Descrizione: Restituisce l'indirizzo del paziente.

Firma: getIndirizzo(self) -> str

■ + getLuogoNascita

Descrizione: Restituisce il luogo di nascita del paziente.

Firma: getLuogoNascita(self) -> str

■ + getMedicId

Descrizione: Restituisce l'ID del medico di riferimento.

Firma: getMedicoId(self) -> int

■ + getMedicoRiferimento

Descrizione: Recupera i dati anagrafici del medico di riferimento del paziente.

Firma: getMedicoRiferimento(self) -> tuple

Ritorna: tuple | None: Tupla nel formato (nome, cognome, email, cel) del medico, oppure None se non trovato.

■ + **getNome**

Descrizione: Restituisce il nome del paziente.

Firma: getNome(self) -> str

■ + **getPuntiGlicemia**

Descrizione: Restituisce i punti gravità legati alla glicemia fuori soglia.

Firma: getPuntiGlicemia(self) -> int

■ + **getPuntiTerapia**

Descrizione: Restituisce i punti gravità legati alla terapia non rispettata.

Firma: getPuntiTerapia(self) -> int

■ + **getRilevazioni**

Descrizione: Restituisce tutte le rilevazioni della glicemia del paziente.

Firma: getRilevazioni(self) -> list

Ritorna: list: Lista di tuple nel formato (giorno (str), ora (str), glicemia (float), primaDopoPasto (str)).

■ + **getSegnalazioni**

Descrizione: Recupera tutte le segnalazioni del paziente.

Firma: getSegnalazioni(self) -> list

Ritorna: list: Lista di tuple nel formato (giorno_inizio (str), giorno_fine (str), descrizione (str), terapia (str|None)).

■ + **getSesso**

Descrizione: Restituisce il sesso del paziente ('M' o 'F').

Firma: getSesso(self) -> str

■ + **getTerapie**

Descrizione: Restituisce tutte le terapie farmacologiche del paziente.

Firma: getTerapie(self) -> list

Ritorna: list: Lista di tuple nel formato (farmaco (str), assunzioniGiornaliere (int), quantita (str), indicazioni (str), id_med (int)).

■ + **getTerapieByName**

Descrizione: Cerca una terapia specifica tramite il nome del farmaco.

Firma: getTerapieByName(self, farmaco: str) -> tuple

Ritorna: farmaco (str): Il nome del farmaco da cercare. tuple | None: Tupla della terapia se trovata, altrimenti None.

■ Modulo: **medico.py**

Percorso: `src/medico.py`

■■ Classe: **Medico**

Descrizione: Rappresenta un medico e fornisce metodi per gestire i suoi pazienti. Args: id_ref (int): L'ID del medico (corrisponde a `id\_med` nella tabella MEDICO). db_path (str, optional): Percorso del file del database SQLite. Default: "database/glicocare.db".

■ **Attributi (Variabili di istanza):**

Nome	Tipo suggerito	Descrizione
id_ref	Any	L'ID del medico (corrisponde a `id_med` nella tabella MEDICO).
db_path	Any	Percorso del file del database SQLite.

■ **Costruttore:**

+ **Medico(self, id_ref, db_path='database/glicocare.db')**

Descrizione: Initialize self. See help(type(self)) for accurate signature.

■ **Metodi (UML: + pubblico, - privato):**

■ **- __str__**

Descrizione: Restituisce una rappresentazione leggibile del medico.

Firma: __str__(self) -> str

■ **- _carica_dati**

Descrizione: Carica i dati anagrafici del medico e la lista dei suoi pazienti dal DB.

Firma: _carica_dati(self)

■ **+ getCel**

Descrizione: Restituisce il numero di cellulare del medico.

Firma: getCel(self) -> str

■ **+ getCf**

Descrizione: Restituisce il codice fiscale del medico.

Firma: getCf(self) -> str

■ **+ getCognome**

Descrizione: Restituisce il cognome del medico.

Firma: getCognome(self) -> str

■ **+ getDataNascita**

Descrizione: Restituisce la data di nascita del medico.

Firma: getDataNascita(self) -> str

■ **+ getEmail**

Descrizione: Restituisce l'email del medico.

Firma: getEmail(self) -> str

■ **+ getIdRef**

Descrizione: Restituisce l'ID di riferimento del medico.

Firma: getIdRef(self) -> int

■ **+ getIndirizzo**

Descrizione: Restituisce l'indirizzo del medico.

Firma: getIndirizzo(self) -> str

■ + getLuogoNascita

Descrizione: Restituisce il luogo di nascita del medico.

Firma: getLuogoNascita(self) -> str

■ + getNome

Descrizione: Restituisce il nome del medico.

Firma: getNome(self) -> str

■ + getPazienteById

Descrizione: Recupera un paziente specifico tramite il suo ID.

Firma: getPazienteById(self, id_paz: int)

Ritorna: id_paz (int): ID del paziente. Paziente | None: Oggetto Paziente, o None se non assegnato a questo medico.

■ + getPazienti

Descrizione: Restituisce la lista di tutti i pazienti come oggetti Paziente, ordinati per gravità decrescente.

Firma: getPazienti(self) -> list

■ + getPazientiConDettaglio

Descrizione: Restituisce una lista di dizionari con i dettagli di ogni paziente, ordinati per gravità decrescente.

Firma: getPazientiConDettaglio(self) -> list

Ritorna: list: Lista di dict ordinata per gravità decrescente.

■ + getPazientIds

Descrizione: Restituisce la lista degli ID dei pazienti assegnati, ordinati per gravità decrescente.

Firma: getPazientiIds(self) -> list

Ritorna: list: Lista di int (id_paz).

■ + getSesso

Descrizione: Restituisce il sesso del medico ('M' o 'F').

Firma: getSesso(self) -> str

■ + getStatistiche

Descrizione: Calcola le statistiche generali dei pazienti del medico.

Firma: getStatistiche(self) -> dict

Ritorna: dict: Dizionario con chiavi: totale, in_regola, gravi, solo_glicemia, solo_terapia, entrambi, gravita_media

■ Modulo: graph_utils.py

Percorso: `src/graph\_utils.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: dashboard_paziente.py

Percorso: `ui/dashboard\_paziente.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: registrazione.py

Percorso: `ui/registrazione.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: assunzioni.py

Percorso: `ui/assunzioni.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: sintomi.py

Percorso: `ui/sintomi.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: dashboard_medico.py

Percorso: `ui/dashboard\_medico.py`

Nessuna classe pubblica definita in questo modulo.

--- Fine Specifiche UML ---